DEEP LEARNING — 046211
FINAL EXAM A
24/2/2025

EXAM DURATION: 3 HOURS

INSTRUCTIONS.

1. You can use a calculator and the 2 formula sheets (4 pages).
2. You need to answer all question parts, unless you received a special permit.
3. The weight of each question is written in the form. The total number of points is 100 points.
4. Please write clearly and orderly. You can write either in English or in Hebrew.
5. You must give the full details of the solution derivation. Solutions without explanations will not be given a score, unless written otherwise.
6. Once you finish the exam, you need to use Gradescope to upload your exam. In addition, you need to submit the notebook.

1. Invariance and Equivariance in Linear Models

Consider a linear regression problem with a neural network of the form $\hat{y} = \mathbf{x}^T\mathbf{w}$, where $\mathbf{x}, \mathbf{w} \in \mathbb{R}^d$. Assume the network is trained to minimize

$$\mathcal{L}_0(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\left(y^{(n)} - \mathbf{w}^T\mathbf{x}^{(n)}\right)^2 = \frac{1}{2}||y - \mathbf{X}\mathbf{w}||^2,$$

where $\mathbf{X} = [\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N)}]^T$ is the sample set and $y = [y^{(1)}, \ldots, y^{(N)}]^T$ is the corresponding label vector.

We define operator $\mathbf{G}$ as some operator on the input $\mathbf{x}$, e.g. the input is an image and $\mathbf{G}$ spins it clockwise 90°. $\mathbf{G}$ is defined via an **invertible** $d \times d$ square matrix.

We define a function f as **invariant** w.r.t. G if $f(\mathbf{G}\mathbf{x}) = f(\mathbf{x})$, i.e. applying $\mathbf{G}$ does not change the output of f.

We will mark $\mathcal{G}$ a set of operators, and consider functions that are invariant w.r.t. all operators in $\mathcal{G}$. For example, if $\mathcal{G}$ is a set of clockwise rotation operators, an invariant f might be the sum of all pixel values.

The following are a set of properties that apply to rotation matrices (operators):

1. If $\mathbf{G} \in \mathcal{G}$, $\mathbf{G}^T \in \mathcal{G}$.
2. Any $\mathbf{G} \in \mathcal{G}$ is **orthogonal** ($\mathbf{G}\mathbf{G}^T = \mathbf{G}^T\mathbf{G} = \mathbf{I}$).
3. Applying operator $\mathbf{G} \in \mathcal{G}$ on all elements of $\mathcal{G}$ returns $\mathcal{G}$, meaning $\bigcup_{\mathbf{G}' \in \mathcal{G}} \mathbf{G}\mathbf{G}' = \mathcal{G}$.

We define the following cost function

$$\mathcal{L}_1(\mathbf{w}) = \frac{1}{2}\sum_{\mathbf{G}\in\mathcal{G}}\left|\left|y - \mathbf{X}\mathbf{G}^T\mathbf{w}\right|\right|^2$$

(1) Explain intuitively why this cost function encourages the predictor $\mathbf{w}$ to be invariant.

(2) Show that $\mathcal{L}_1(\mathbf{w})$ is invariant to rotation transformations of the form $\mathbf{w} \to \mathbf{G}\mathbf{w}$, for all $\mathbf{G} \in \mathcal{G}$.

(3) Prove that the optimal $\mathbf{w}_1^*$ that minimizes $\mathcal{L}_1(\mathbf{w})$ is

$$\mathbf{w}_1^* = \mathbf{K}^{-1}\mathbf{R}\mathbf{X}^T\mathbf{Y},$$

where $\mathbf{K} = \sum_{\mathbf{G}\in\mathcal{G}} \mathbf{G}^T\mathbf{X}^T\mathbf{X}\mathbf{G}, \mathbf{R} = \sum_{\mathbf{G}\in\mathcal{G}} \mathbf{G}$ and we assume $\mathbf{K}$ is invertible.

(4) Show that $\mathbf{G}\mathbf{R} = \mathbf{R}$ for all $\mathbf{G} \in \mathcal{G}$.

(5) Show that $\mathbf{G}\mathbf{K}^{-1}\mathbf{G}^T = \mathbf{K}^{-1}$ for all $\mathbf{G} \in \mathcal{G}$.

Hint: for invertible $\mathbf{A}, \mathbf{B}$, $(\mathbf{A}\mathbf{B})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$

(6) Prove that the optimal $\mathbf{w}_1^*$ yields a predictor $\hat{y}(\mathbf{x}) = \mathbf{x}^T\mathbf{w}_1^*$ that is invariant to $\mathbf{x} \to \mathbf{G}\mathbf{x}$ for all $\mathbf{G} \in \mathcal{G}$.

Suppose that the network is now of the form $\hat{\mathbf{y}} = \mathbf{W}\mathbf{x}$, where $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{W} \in \mathbb{R}^{d \times d}$, $\hat{\mathbf{y}} \in \mathbb{R}^d$.

A function f is **equivariant** to operator $\mathbf{G}$ if $f(\mathbf{G}\mathbf{x}) = \mathbf{G}f(\mathbf{x})$, i.e. applying the operator on the input is the same as applying the operator on the output.

Let $\mathbf{X} = [\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N)}]^T$ be the sample set and $\mathbf{Y} = [\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(N)}]^T$ is the corresponding label matrix.

(7) Suggest a cost function $\mathcal{L}_2(\mathbf{w})$ that encourages the predictor $\hat{y} = \mathbf{W}\mathbf{x}$ is equivariant for all $\mathbf{G} \in \mathcal{G}$. Provide a short explanation.

2. Batch Size and Step Size in SGD

Let $\mathcal{L}$ be a loss function which can be decomposed into N parts $\ell^{(n)}$

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N}\sum_{n=1}^{N} \ell^{(n)}(\mathbf{w}) \ .$$

We examine minibatch-stochastic gradient descent training

$$\hat{\boldsymbol{g}}_t = \frac{1}{B}\sum_{n\in\mathcal{B}} \nabla\ell^{(n)}(\mathbf{w}_t)$$
$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta\hat{\boldsymbol{g}}_t$$

where $\hat{\boldsymbol{g}}_t$ is gradient estimate on the minibatch, $\mathcal{B}$ is a randomly selected subset of $\{1,\ldots,N\}$ of size B, and η is the step size. We wish to understand how to select the step size η given the batch size B. For simplicity, we approximate the loss change after a single step using a second-order Taylor function

$$\mathcal{L}(\mathbf{w}_t - \eta\hat{\boldsymbol{g}}_t) \approx \mathcal{L}(\mathbf{w}_t) - \eta\hat{\boldsymbol{g}}_t^\top \boldsymbol{g}_t + \frac{1}{2}\eta^2\hat{\boldsymbol{g}}_t^\top \mathbf{H}_t\hat{\boldsymbol{g}}_t \tag{2.1}$$

where $\boldsymbol{g}_t = \nabla\mathcal{L}(\mathbf{w}_t) = \mathbb{E}\hat{\boldsymbol{g}}_t$ and $\mathbf{H}_t = \nabla^2\mathcal{L}(\mathbf{w}_t)$.

(1) For full batch gradient descent, i.e. $B = N$, what is the optimal step size η which minimize the loss in Eq. 2.1?

(2) Show that for a general batch size

$$\mathbb{E}\mathcal{L}(\mathbf{w}_t - \eta\hat{\boldsymbol{g}}_t) = \mathcal{L}(\mathbf{w}_t) - \eta a_t + \frac{1}{2}\eta^2\left(b_t + \frac{c_t}{B}\right) \tag{2.2}$$

and write a_t, b_t and c_t explicitly as functions of $\boldsymbol{g}_t, \mathbf{H}_t$ and $\boldsymbol{\Sigma}_t = \mathbb{E}\left(\hat{\boldsymbol{g}}_t\hat{\boldsymbol{g}}_t^\top\right) - \boldsymbol{g}_t\boldsymbol{g}_t^\top$, i.e. the covariance matrix of $\hat{\boldsymbol{g}}_t$. Hint: for any vector $\mathbf{v}$ and matrix $\mathbf{A}$, we have

$$\mathbf{v}^\top\mathbf{A}\mathbf{v} = \mathrm{Tr}\left[\mathbf{v}^\top\mathbf{A}\mathbf{v}\right] = \mathrm{Tr}\left[\mathbf{A}\mathbf{v}\mathbf{v}^\top\right] \ .$$

(3) Is it feasible to calculate a_t, b_t and c_t directly? Suggest another method to efficiently calculate a_t, b_t and c_t without using second-order derivatives.

For simplicity, from now on, we assume that: $a_t = a, b_t = b$ and $c_t = c$ are constant.

(4) What is the optimal step size η_* which minimizes the loss in Eq. 2.1? Does this approximately match what we saw in class empirically (consider two cases: very large B and very small B)? Explain.

(5) What is the loss decrease after a taking a step with the optimal step size, i.e. $\mathbb{E}\mathcal{L}(\mathbf{w}_t - \eta_*\hat{\boldsymbol{g}}_t) - \mathcal{L}(\mathbf{w}_t)$? Does this approximately match what we saw in class empirically (consider two cases: very large B and very small B)? Explain.

3. Masking in Transformers

We examine the classic encoder-decoder transformer model for next-token prediction, and focus on its masking mechanism.

(1) Draw a picture of the , and qualitatively explain all its parts.

(2) Why do we need "Masked self-attention" in the decoder, instead of the regular self attention? Give an example with a specific task.

(3) To implement Multi-Head Masked Self-Attention (MHMSA) in the decoder we use

$$\forall i \in \{1,\ldots,H\} : \mathbf{Q}_i = \mathbf{X}\mathbf{W}_i^Q, \mathbf{K}_i = \mathbf{X}\mathbf{W}_i^K, \mathbf{V}_i = \mathbf{X}\mathbf{W}_i^V$$

$$\text{MHMSA}\left(\mathbf{X}\right) = \sum_{i=1}^{H} \text{Softmax}\left(\frac{\mathbf{Q}_i\mathbf{K}_i^\top}{\sqrt{d}} + \mathbf{M}\right)\mathbf{V}_i\mathbf{W}_i^O \in \mathbb{R}^{T\times d}.$$

Explain how should we choose $\mathbf{M} \in \mathbb{R}^{T\times T}$ to implement masking based on the expression for MHMSA above, and why this works.

(4) Why do we not require masking in the Multi-Head Masked Self-Attention (MHSA) right after the MHMSA? Base this on the expression for MHSA.

(5) Explain the difference in operation of the MHMSA in training and inference. Which method is faster?

(6) A researcher though it might be useful if (similarly to vision models) we add a "max-pooling" with pooling size 2 layer on the token dimension at the end of the first decoder blocks (after the fully connected layers), to reduce its size from T to $T/2$, and then extend it back to the original T (e.g., by replicating the tokens) before the last decoder block. Explain why is this problematic (hint: it is related to the reason we need masking), and how to fix the issue above, allowing this max-pooling layer to be added.

Solution of Question 1: Invariance and Equivariance in Linear Models

See solution of Question 2 in Spring 2024, Exam B.

Solution of Question 2: Batch Size and Step Size in SGD

The solution is based on section 2.2 here: `https://arxiv.org/abs/1812.06162`.

1. In this case

$$\mathcal{L}\left(\mathbf{w}_t - \eta \boldsymbol{g}_t\right) \approx \mathcal{L}\left(\mathbf{w}_t\right) - \eta \left\|\boldsymbol{g}_t\right\|^2 + \frac{1}{2}\eta^2 \boldsymbol{g}_t^\top \mathbf{H}_t \boldsymbol{g}_t$$

differentiating with respect to η, and equating to zero we get

$$0 = -\left\|\boldsymbol{g}_t\right\|^2 + \eta \boldsymbol{g}_t^\top \mathbf{H}_t \boldsymbol{g}_t$$

so

$$\eta = \frac{\left\|\boldsymbol{g}_t\right\|^2}{\boldsymbol{g}_t^\top \mathbf{H}_t \boldsymbol{g}_t}$$

2. In this case

$$\mathbb{E}\mathcal{L}\left(\mathbf{w}_t - \eta \hat{\boldsymbol{g}}_t\right) \approx \mathcal{L}\left(\mathbf{w}_t\right) - \eta \mathbb{E}\left[\hat{\boldsymbol{g}}_t^\top \boldsymbol{g}_t\right] + \frac{1}{2}\eta^2 \mathbb{E}\left[\hat{\boldsymbol{g}}_t^\top \mathbf{H}_t \hat{\boldsymbol{g}}_t\right]$$

We have that

$$\mathbb{E}\hat{\boldsymbol{g}}_t^\top \boldsymbol{g}_t = \boldsymbol{g}_t^\top \boldsymbol{g}_t = \left\|\boldsymbol{g}_t\right\|^2$$

and

$$\begin{aligned}
\mathbb{E}\left[\hat{\boldsymbol{g}}_t^\top \mathbf{H}_t \hat{\boldsymbol{g}}_t\right] &= \mathbb{E}\mathrm{Tr}\left(\hat{\boldsymbol{g}}_t^\top \mathbf{H}_t \hat{\boldsymbol{g}}_t\right) = \mathbb{E}\mathrm{Tr}\left(\mathbf{H}_t \hat{\boldsymbol{g}}_t \hat{\boldsymbol{g}}_t^\top\right) \\
&= \mathrm{Tr}\left(\mathbf{H}_t \mathbb{E}\left[\hat{\boldsymbol{g}}_t \hat{\boldsymbol{g}}_t^\top\right]\right) = \mathrm{Tr}\left(\mathbf{H}_t\left(\boldsymbol{\Sigma}_t + \boldsymbol{g}_t \boldsymbol{g}_t^\top\right)\right) \\
&= \mathrm{Tr}\left(\mathbf{H}_t \boldsymbol{\Sigma}_t + \mathbf{H}_t \boldsymbol{g}_t \boldsymbol{g}_t^\top\right) = \mathrm{Tr}\left(\mathbf{H}_t \boldsymbol{\Sigma}_t + \boldsymbol{g}_t^\top \mathbf{H}_t \boldsymbol{g}_t\right) \\
&= \boldsymbol{g}_t^\top \mathbf{H}_t \boldsymbol{g}_t + \mathrm{Tr}\left(\mathbf{H}_t \boldsymbol{\Sigma}_t\right)
\end{aligned}$$

Therefore,

$$\mathbb{E}\mathcal{L}\left(\mathbf{w}_t - \eta \hat{\boldsymbol{g}}_t\right) = \mathcal{L}\left(\mathbf{w}_t\right) - \eta \left\|\boldsymbol{g}_t\right\|^2 + \frac{1}{2}\eta^2 \left(\boldsymbol{g}_t^\top \mathbf{H}_t \boldsymbol{g}_t + \frac{1}{B}\mathrm{Tr}\left(\mathbf{H}_t \boldsymbol{\Sigma}_t\right)\right)$$

therefore, with $a_t = \left\|\boldsymbol{g}_t\right\|^2$, $b_t = \boldsymbol{g}_t^\top \mathbf{H}_t \boldsymbol{g}_t$, and $c_t = \mathrm{Tr}\left(\mathbf{H}_t \boldsymbol{\Sigma}_t\right)$.

3. Calculating $a_t = \left\|\boldsymbol{g}_t\right\|^2$ is easy, since it requires only first order derivatives. Calculating $b_t = \boldsymbol{g}_t^\top \mathbf{H}_t \boldsymbol{g}_t$ is harder, but doable, since it involves a Hessian vector product $\mathbf{H}_t \boldsymbol{g}_t$ which can be done by Backpropagation through the gradients, since $\mathbf{H}_t \boldsymbol{g}_t = \nabla\left(\frac{1}{2}\left\|\boldsymbol{g}_t\right\|^2\right)$. Calculating $c_t = \mathrm{Tr}\left(\mathbf{H}_t \boldsymbol{\Sigma}_t\right)$ is unfeasible if we want a precise results, since even storing the matrix $\boldsymbol{\Sigma}_t$ is impossible. However, we can simply scan over several values of η and B and measure $\mathbb{E}\mathcal{L}\left(\mathbf{w}_t - \eta \hat{\boldsymbol{g}}_t\right) - \mathcal{L}\left(\mathbf{w}_t\right)$ to find a set of linear equations from which we can extract the values of a_t, b_t, and c_t. Assuming the equations are not singular (which is the generic case), we only need three equations.

4. Taking the derivative of 2.2 with respect to η and equating to zero, we get

$$\eta_* = \frac{a}{b + \frac{c}{B}}\,.$$

Note that

$$\eta_* = \begin{cases} B\frac{a}{c} & B \ll \frac{c}{b} \\ \frac{a}{b} & B \gg \frac{c}{b} \end{cases}\,.$$

This roughly matches what we saw in the training methods lecture, that

$$\eta_* \approx \begin{cases} q_1 B^\alpha & B < B_* \\ q_2 & B > B_* \end{cases},$$

where q_1, q_2, B_* and $\alpha \in [0.5, 1]$ are some constants, when $\alpha = 1$.

5. Plugging the solution from the previous part

$$\mathbb{E}\mathcal{L}\left(\mathbf{w}_t - \eta_* \hat{\boldsymbol{g}}_t\right) - \mathcal{L}\left(\mathbf{w}_t\right) = -\frac{1}{2}\frac{a^2}{b + \frac{c}{B}}.$$

Note that

$$\mathbb{E}\mathcal{L}\left(\mathbf{w}_t - \eta \hat{\boldsymbol{g}}_t\right) - \mathcal{L}\left(\mathbf{w}_t\right) = -\frac{1}{2}\begin{cases} B\frac{a^2}{c} & B \ll \frac{c}{b} \\ \frac{a^2}{b} & B \gg \frac{c}{b} \end{cases}.$$

So the decrease in the loss after T steps is

$$\begin{aligned} \sum_{t=1}^{T}\left(\mathbb{E}\mathcal{L}\left(\mathbf{w}_t - \eta \hat{\boldsymbol{g}}_t\right) - \mathcal{L}\left(\mathbf{w}_t\right)\right) &= -\frac{1}{2}\begin{cases} B\sum_{t=1}^{T}\frac{a^2}{c} & B \ll \frac{c}{b} \\ \sum_{t=1}^{T}\frac{a^2}{b} & B \gg \frac{c}{b} \end{cases} \\ &= -\frac{1}{2}\begin{cases} BT\frac{a^2}{c} & B \ll \frac{c}{b} \\ T\frac{a^2}{b} & B \gg \frac{c}{b} \end{cases}. \end{aligned}$$

where in the last line we assumed that a_t, b_t and c_t are constant. So, to decrease the loss by some amount $\Delta\mathcal{L}$, we need

$$\Delta\mathcal{L} = \frac{1}{2}\begin{cases} BT\frac{a^2}{c} & B \ll \frac{c}{b} \\ T\frac{a^2}{b} & B \gg \frac{c}{b} \end{cases}$$

i.e.

$$T = \begin{cases} \frac{1}{B}\frac{2c\Delta\mathcal{L}}{a^2} & B \ll \frac{c}{b} \\ \frac{2b\Delta\mathcal{L}}{a^2} & B \gg \frac{c}{b} \end{cases}$$

This approximately matches what we saw in the training methods lecture, that

$$T \approx \begin{cases} \frac{s_1}{B} & B < B_* \\ s_2 & B > B_* \end{cases},$$

where s_1, s_2 and B_* are some constants.

Solution of Question 3: Masking in Transformers

1. See lecture for the general model description.

2. We need masked self-attention to ensure the model does not have access to "illegal" (non-casual) tokens during training. For example, if we want to translate text from a source to target language, and in each token t he transformer should predict the next token $t+1$, then the masking prevent access to future words in the target task (including the $t+1$ word, which we we want to find).

3. For each token in the output of the MHMSA layer

$$\text{MHMSA}\left(\mathbf{X}\right)[t,:] = \sum_{i=1}^{H}\sum_{t'=1}^{T} \frac{\exp\left(\frac{1}{\sqrt{d}}\mathbf{Q}_i[t,:]\mathbf{K}_i^\top[:,t'] + \mathbf{M}[t,t']\right)}{\sum_{u=1}^{T}\exp\left(\frac{1}{\sqrt{d}}\mathbf{Q}_i[t,:]\mathbf{K}_i^\top[:,u] + \mathbf{M}[t,u]\right)}\mathbf{V}_i[t',:]\mathbf{W}_i^O\,.$$

In the problem of next-token prediction, we want token t not to be affected by any token $t' > t$. Therefore, $\forall t' > t$ we set $\mathbf{M}[t,t'] = -c$ for some large c, and $\mathbf{M}[t,t'] = 0$ otherwise. In this case

$$\forall t' > t : \lim_{c\to\infty} \exp\left(\frac{1}{\sqrt{d}}\mathbf{Q}_i[t,:]\mathbf{K}_i^\top[:,t'] - c\right) = 0$$

and all other tokens will not be affected, so in this limit we get

$$\text{MHMSA}\left(\mathbf{X}\right)[t,:] = \sum_{i=1}^{H}\sum_{t'=1}^{t} \frac{\exp\left(\frac{1}{\sqrt{d}}\mathbf{Q}_i[t,:]\mathbf{K}_i^\top[:,t'] + \mathbf{M}[t,t']\right)}{\sum_{u=1}^{t}\exp\left(\frac{1}{\sqrt{d}}\mathbf{Q}_i[t,:]\mathbf{K}_i^\top[:,u] + \mathbf{M}[t,u]\right)}\mathbf{V}_i[t',:]\mathbf{W}_i^O$$

which means $\text{MHMSA}\left(\mathbf{X}\right)[t,:]$ does not include any inputs from time $t' > t$, as required.

4. In the Multi-Head Self-Attention after the Multi-Head Masked Self-Attention in the decoder, the keys and values are taken from the encoder where it is OK to use future tokens (e.g., text from future words in the source language), while only the queries are taken from the decoder input (e.g., the target language). Therefore, if we examine token t in the MHSA ouput,

$$\text{MMSA}\left(\mathbf{X}\right)[t,:] = \sum_{i=1}^{H}\sum_{t'=1}^{T} \frac{\exp\left(\frac{1}{\sqrt{d}}\mathbf{Q}_i[t,:]\mathbf{K}_i^\top[:,t']\right)}{\sum_{u=1}^{T}\exp\left(\frac{1}{\sqrt{d}}\mathbf{Q}_i[t,:]\mathbf{K}_i^\top[:,u]\right)}\mathbf{V}_i[t',:]\mathbf{W}_i^O$$

we observe it only depends on token t from the decoder (through $\mathbf{Q}_i[t,:]$) and all tokens from the encoder (through $\mathbf{K}_i^\top[:,t']$ and $\mathbf{V}_i[t',:]$). Therefore, we do not use "illegal" future information from the decoder here.

5. During training, the transformers operates simultaneously (in parallel) on all the tokens $\{1,\ldots,T\}$, since we can use the masking to avoid any "illegal" interactions between the tokens, as described above. During inference, the output is generated token by token (e.g., each time we only output the next word in the target language sentence). Therefore, we can pass the input through the encoder in parallel, the decoder must be used iteratively, producing token by token. Note that the decoder uses masking during inference to be similar to its operation during training, even though it does not need it.

6. The problem is that the max-pool operation mixes tokens from different times, so in the end the token t can be affected by token $t+1$ in the decoder input. To make sure this does not happen, we can shift token dimension by 1: $t \to t-1$ before we use the max-pooling operation.